

eParts Services

ML Confidence Scoring System Technical Proposal & Data Requirements

Prepared by: MSE Studio Team

Date: March 2026

Version: 1.0

1. Executive Summary

We are building the Attribute Prediction Service — the ML component inside the eParts reference architecture. Its role is to take product data that has already been ingested and staged, predict the correct attribute values for each product, and attach a confidence score to every prediction.

Based on that score, the system automatically routes each item:

- High confidence → Auto-Accept + Writeback to PIMS (no human needed).
- Low confidence → Human Review Queue, with a clear explanation of why confidence is low.

The eParts ops team only handles the hard or ambiguous cases. Routine submissions are processed instantly. The model improves over time through manually-triggered retraining cycles informed by reviewer feedback.

Key concept — Confidence Score:

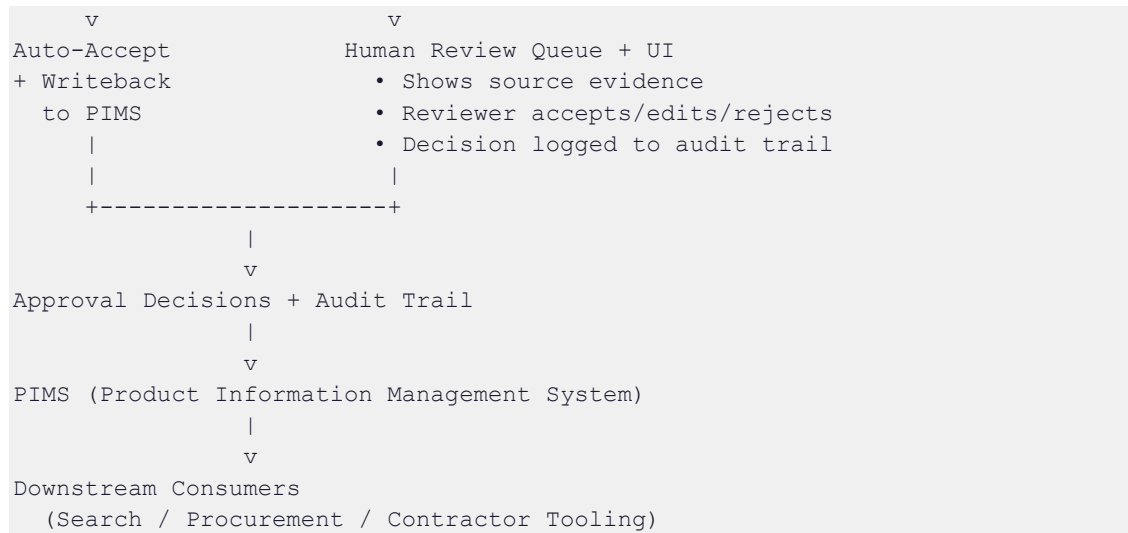
Every prediction comes with a number between 0 and 1 (0%–100%). A score of 0.90 means the model is 90% confident in that attribute value. Thresholds for auto-accept vs. human review are configurable by ops leads, and every threshold change is logged per the audit requirements defined in Section 6.8.

2. Where the ML Service Fits in the eParts Architecture

The eParts reference architecture defines the full pipeline from supplier intake to downstream consumers. The ML service operates specifically on the output of the Intermediate Structured Layer (Staging / Canonical Tables) and feeds into the Confidence Routing component.



```
Suppliers / Manufacturers
  (Email, SFTP, CSV, PDFs, websites)
  |
  v
Ingestion Gateway
  • Validates, logs, assigns submission ID, stores raw file
  |
  v
Intermediate Structured Layer (Staging / Canonical Tables)
  • Converts raw input to standard column format
  • Stores parsed fields separately from predictions
  • Tracks lineage: raw → extracted → normalized → predicted → writeback
  |
  v
>>> ATTRIBUTE PREDICTION SERVICE [THIS DOCUMENT] <<<
  • Rule engine + ML model → predicted attribute values
  • Confidence score per attribute
  • Records model version for every prediction
  |
  [split by confidence score]
  |
```



The cross-cutting components (Logs, Metrics, Dashboards / Alerts) monitor every step of this pipeline, including model performance and queue length.

3. What the Attribute Prediction Service Does — Plain Language

Think of the service as a well-trained catalog specialist who has studied the entire eParts product catalog. When a supplier submits a new product spec sheet, the specialist:

1. 1. Receives the pre-processed, structured row from the Staging Table.
2. 2. Checks known patterns first — part numbers, voltage ratings, manufacturer names — using a fast rule engine.
3. 3. For anything the rules cannot resolve, compares the product description against all known correct examples to find the closest match.
4. 4. Rates confidence: “I’m 93% sure the OUTPUT SIGNAL is 0–10V” vs. “I’m only 48% sure — please review, here is why.”
5. 5. Sends high-confidence results straight to PIMS; sends low-confidence results to the review queue with an explanation.
6. 6. When a reviewer corrects a mistake, the feedback is recorded. At the next manually-triggered retraining cycle, the model is updated and promoted through the model registry.

Important constraint (from the eParts architecture requirements, §6.4):

Retraining must be manually triggered — fully automated retraining is not allowed. This keeps ops teams in control of which model version is live, and ensures predictions remain reproducible: the same model version + same input always gives the same output.

4. Technical Design with Key Formulas

Each formula below is followed by a plain-language explanation. You do not need to understand the formulas to use or evaluate this system.

4.1 Layer 1: Rule Engine (fast, high-confidence cases)

The rule engine handles structured, unambiguous inputs first — before any ML is involved. It uses pattern matching against known catalog data.

Examples of rule patterns:

Input Pattern (example)	Mapped Output
'AIM2'	Product_ID lookup in existing catalog
'24 VAC (+/- 10%)'	SUPPLY_VOLTAGE = '24VAC'
'4-20 mA'	INPUT_SIGNAL = '4-20mA'
'-20°C to 70°C'	TEMPERATURE_RANGE = '-20 TO 70°C'
'Automation Components, Inc.'	Manufacturer_ID lookup

Confidence assigned by the rule engine:

Match Type	conf_rule Value
Exact catalog match (part number found)	1.00
Numeric + unit pattern match	0.90
Partial / keyword match	0.65 – 0.80
No match — pass to ML layer	0.00

4.2 Layer 2: Text-to-Vector Conversion

Plain language: Computers cannot directly compare words. We first convert product description text into a list of numbers (a 'vector') that captures its meaning. Two descriptions with similar meanings produce similar vectors. Technical terms like 'thermistor' or 'Rogowski coil' are weighted more heavily than common words like 'the' or 'for'.

Formula — TF-IDF Weighted Sentence Vector

$$\mathbf{v}(\text{text}) = \frac{\sum_{i=1}^n (\text{tfidf}(w_i) \cdot \text{embed}(w_i))}{\sum_{i=1}^n \text{tfidf}(w_i)}$$

Where:

- w_i : the i -th word in the text snippet
- $\text{embed}(w_i)$: the pre-trained embedding vector for word w_i
 - a numeric vector (for example, 100 dimensions) that represents the semantic meaning of the word
- $\text{tfidf}(w_i)$: the importance weight of word w_i , defined as

$$\text{tfidf}(w_i) = \text{TF}(w_i) \times \text{IDF}(w_i)$$

- $\text{TF}(w_i)$: term frequency of word w_i in the current text snippet
- $\text{IDF}(w_i)$: inverse document frequency of word w_i , defined as

$$\text{IDF}(w_i) = \log \left(\frac{\text{total number of documents}}{\text{number of documents containing } w_i} \right)$$

Interpretation:

This formula builds a single vector representation for the whole text snippet by taking a weighted average of its word embeddings. Words that are more informative in the domain receive higher weight, while very common words receive little or almost no weight.

For example:

- domain-specific words such as “**Rogowski**,” “**thermistor**,” and “**VAC**” receive **high weight**
- very common words such as “**the**,” “**and**,” and “**for**” receive **near-zero weight**

4.3 Layer 2: Similarity Measurement

Plain language: We measure how 'close' two vectors are using cosine similarity, which measures the angle between them. An angle of 0° means identical meaning; 90° means completely unrelated. This works better than straight-line distance for high-dimensional text data.

Formula – Similarity Between Two Text Vectors

Given two text vectors A and B , their similarity is measured using **cosine similarity**:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where:

- $A \cdot B$: the dot product of vectors A and B
- $\|A\|$: the magnitude (length) of vector A
- $\|B\|$: the magnitude (length) of vector B

Interpretation:

Cosine similarity measures how aligned the two vectors are.

- a value close to **1** means the two texts are very similar
- a value close to **0** means they are weakly related
- a value close to **-1** means they are strongly dissimilar or opposite in direction

To make the score easier to use in downstream logic, it can be rescaled from the range $[-1, 1]$ to $[0, 1]$:

$$\text{sim} = \frac{\text{cosine_value} + 1}{2}$$

Rescaled interpretation:

- **1.0** = highest similarity

- 0.5 = neutral / weak similarity
- 0.0 = lowest similarity

Interpretation:

Score	Meaning
sim = 1.0	Identical meaning
sim ≥ 0.8	Closely related — typically safe to auto-accept
sim = 0.5	Weakly related — route to review queue
sim ≤ 0.3	Unrelated — flag as unable to process

4.4 Layer 2: Reference Region and Confidence Score

Plain language: We compute the average vector of all confirmed-correct historical examples — called the centroid. This is the 'center of gravity' of what correct product attribute entries look like. The further a new input is from this center, the lower the confidence score.

Formula — Centroid of Reference Examples

Let N be the number of confirmed-correct training examples. The centroid of the reference set is:

$$\mu = \frac{1}{N} \sum_{i=1}^N \mathbf{v}(\text{text}_i)$$

Where:

- μ : the centroid vector of the confirmed-correct reference examples
- N : the number of confirmed-correct training examples
- $\mathbf{v}(\text{text}_i)$: the vector representation of the i -th training text

Interpretation:

This centroid is the average vector of all confirmed-correct examples. It represents the “center” of the known-good reference cluster in embedding space.

Formula — Mahalanobis Distance (accounts for data distribution shape):

To measure how far an incoming product text is from the reference cluster while accounting for the shape of the data distribution, use the Mahalanobis distance:

$$D(q) = \sqrt{(q - \mu)^T \Sigma^{-1} (q - \mu)}$$

Where:

- q : the vector of the incoming product text
- μ : the centroid of the confirmed-correct training examples
- Σ : the covariance matrix of the reference examples

- Σ^{-1} : the inverse of the covariance matrix
- $(q-\mu)^T$: the transpose of the difference vector

Interpretation:

Mahalanobis distance measures how unusual the incoming text vector is relative to the confirmed-correct reference set.

Unlike ordinary Euclidean distance, it takes into account the **shape**, **spread**, and **direction** of the reference data distribution.

That means:

- if q lies close to the normal reference cluster, then $D(q)$ is **small**
- if q is far from the typical reference pattern, then $D(q)$ is **large**

This makes it useful for identifying whether a new product description looks consistent with known-good examples or appears abnormal.

Why not straight-line distance? Confirmed correct examples for, say, 'current transformers' may naturally form an elongated cluster in vector space (wide variation in current range, narrow variation in accuracy spec). Mahalanobis distance accounts for this shape; straight-line distance does not, and would unfairly penalize valid submissions.

Formula — Embedding-based confidence score:

$$\text{conf}_{\text{embed}} = \exp \left(-\frac{D(q)^2}{2\sigma^2} \right)$$

Where:

- $\text{conf}_{\text{embed}}$: the embedding-based confidence score
- $D(q)$: the Mahalanobis distance of the incoming product text vector from the reference cluster
- q : the vector of the incoming product text
- σ : a scale parameter used to control how quickly confidence decreases as distance increases

Interpretation:

This formula converts the distance from the reference cluster into a confidence score in the range (0,1].

- when $D(q)$ is close to **0**, the input is very close to the center of known correct examples, so $\text{conf}_{\text{embed}}$ is close to **1.0**
- when $D(q)$ is large, the input is far from the known correct examples, so $\text{conf}_{\text{embed}}$ moves closer to **0.0**

In other words:

- **small distance → high confidence**
- **large distance → low confidence**

Calibration note:

The parameter σ is calibrated on a held-out validation set during each training cycle so that the confidence score better matches the actual distribution of known-correct examples.

4.5 k-Nearest Neighbors Scoring (Simpler Alternative)

*Plain language: Find the k most similar confirmed-correct examples and average their similarity scores.
Easier to explain to reviewers: 'The 5 most similar products in the catalog are X, Y, Z — the submitted item is 82% similar on average.'*

Formula — k-Nearest Neighbor Confidence Score

$$\text{conf}_{\text{knn}}(q) = \frac{1}{k} \sum_{i=1}^k \text{sim}(q, v_i)$$

Where:

- $\text{conf}_{\text{knn}}(q)$: the k-nearest-neighbor confidence score for input vector q
- q : the vector of the incoming product text
- v_i : the i -th reference vector among the k most similar reference examples to q
- $\text{sim}(q, v_i)$: the similarity score between q and reference vector v_i
- k : the number of nearest neighbors used in the calculation

Recommended starting value:

$$k = 5$$

Interpretation:

This score is computed as the average similarity between the incoming product vector and its top- k most similar reference examples.

- if the nearest neighbors are highly similar to q , then $\text{conf}_{\text{knn}}(q)$ is **high**
- if even the nearest neighbors are only weakly similar, then $\text{conf}_{\text{knn}}(q)$ is **low**

This provides a simple local confidence estimate based on how closely the input matches known reference examples.

Explainability note:

This method also directly supports explainability requirements because the k nearest neighbors can be shown to the reviewer as evidence for why the system produced a given confidence score or prediction.

4.6 Explainability — Why Is This Item Low Confidence?

Architecture requirement §6.5: 'Keep decisions explainable to ops users; do not hide why something is low confidence.'

Each low-confidence item sent to the Human Review Queue must include a reason. The system generates this automatically from the scoring signals:

Reason Code	Description + Example	Reviewer Action
RULE_NO_MATCH	No rule pattern matched any field. Example: 'Output: 0-1mA sink or source'	Confirm the correct OUTPUT_SIGNAL value.
LOW_SIMILARITY	Closest match in reference catalog is only 51% similar. Nearest known: RCT16-2500 (Rogowski Coil CT)	Confirm whether this is the intended product category.
AMBIGUOUS_VALUE	Multiple valid values match equally well. Candidates: '0-10V' (sim=0.74), '0-5V' (sim=0.71)	Confirm the correct OUTPUT_SIGNAL range.
MISSING_FIELD	Required attribute not found in source text. Missing: MOUNTING_LOCATION	Check original supplier document for this field.

4.7 Final Combined Confidence Score

Formula — Final Combined Confidence Score

$$\text{conf}_{\text{final}} = \alpha \cdot \text{conf}_{\text{rule}} + (1 - \alpha) \cdot \text{conf}_{\text{embed}}$$

Where:

- $\text{conf}_{\text{final}}$: the final confidence score used by the system
- $\text{conf}_{\text{rule}}$: the confidence score from the rule engine, in the range [0, 1]
- $\text{conf}_{\text{embed}}$: the confidence score from semantic similarity or embedding-based matching, in the range [0, 1]
- α : the weighting parameter that controls the contribution of the rule-based score

Initial setting:

$$\alpha = 0.7$$

This gives more weight to the rule engine when rules are available, while still allowing the embedding-based score to influence the final result.

Interpretation:

This formula combines two signals:

- a **rule-based confidence** signal, which is typically stronger when an explicit match or pattern is found
- an **embedding-based confidence** signal, which captures semantic similarity even when no exact rule match exists

The combined score allows the system to balance precision from rules with flexibility from semantic matching.

Routing thresholds (configurable by ops leads; every change is logged per §6.5):

Confidence Range	Action
$\text{conf_final} \geq 0.85$	Auto-Accept + Writeback to PIMS
$0.50 \leq \text{conf_final} < 0.85$	Send to Human Review Queue (reason code attached)
$\text{conf_final} < 0.50$	Flag as 'unable to process' — alert ops team

4.8 Model Versioning and Retraining

Architecture requirements §6.4:

- *ML workflow must support a model registry and controlled promotion.*
- *Retraining must be manually triggered; fully automated retraining is NOT allowed.*
- *Predictions must be reproducible: same model version + same input = same output.*

Each prediction record in the Audit Trail stores the model version that produced it. Retraining follows this controlled process:

Model Versioning and Controlled Retraining Process

Each prediction record stored in the Audit Trail includes the **model version** that produced it. This ensures that every prediction can be traced back to the exact model used at the time of inference.

Retraining is performed through a controlled, manually approved process led by the operations lead.

Retraining Cycle (Manually Triggered by Ops Lead)

1. Collect reviewer corrections since the last training run
2. Merge newly confirmed examples into the training set
3. Train a candidate model as a new version
4. Evaluate the candidate on a held-out validation set; the candidate must outperform the current production model
5. Ops lead reviews the evaluation metrics and approves promotion
6. Promote the candidate to production in the model registry

7. Retain the previous production version for rollback if needed

Interpretation:

This process ensures that model updates are controlled, auditable, and reversible. A new model is not promoted automatically. It must first demonstrate measurable improvement on validation data and then be explicitly approved before deployment.

Formula — Incremental Centroid Update

Between full retraining cycles, the system may apply a lightweight update to the reference centroid used for similarity scoring.

When a reviewer confirms a new correct example, the centroid is updated as:

$$\mu_{\text{new}} = \frac{N\mu_{\text{old}} + v_{\text{new}}}{N + 1}$$

Where:

- μ_{old} : the previous centroid
- μ_{new} : the updated centroid
- N : the number of confirmed examples currently represented in the centroid
- v_{new} : the vector of the newly confirmed correct example

Interpretation:

This formula updates the centroid by incorporating the new confirmed example without requiring a full recomputation from all historical data.

Formula — Lightweight Correction Update

When a reviewer corrects a high-confidence wrong prediction, the centroid can be adjusted as:

$$\mu_{\text{corrected}} = \mu_{\text{old}} - \lambda(v_{\text{wrong}} - \mu_{\text{old}})$$

with

$$\lambda = 0.01$$

Where:

- $\mu_{\text{corrected}}$: the adjusted centroid
- μ_{old} : the previous centroid
- v_{wrong} : the vector associated with the incorrect high-confidence prediction
- λ : a small correction factor controlling the size of the update

Interpretation:

This lightweight correction slightly shifts the centroid away from the vector of a known incorrect high-confidence prediction. The goal is to reduce the chance that similar errors receive overly high similarity scores in the future.

Important Note

This lightweight update affects **similarity-based scoring only**. It does **not** update the weights of the machine learning classifier.

A **full retraining cycle** is still required to:

- update model parameters
- produce a new model version
- validate performance improvements
- promote a new production model

4.9 How Much Training Data Do We Need?

There is a mathematical framework (PAC learning — 'Probably Approximately Correct') that estimates the minimum number of training examples needed to achieve a target accuracy. The estimate below uses the shared Attributes.csv (~200 active attribute types).

Formula — Minimum Number of Labeled Examples

$$m \geq \frac{1}{\varepsilon^2} \ln \left(\frac{|H|}{\delta} \right)$$

Where:

- m : the minimum number of labeled training examples required
- ε : the acceptable error rate
 - example: $\varepsilon = 0.10$ means the system may allow up to **10% error**
- δ : the failure probability
 - example: $\delta = 0.05$ means **95% statistical confidence**
- $|H|$: the size of the hypothesis space
 - in this design, approximated by the number of attribute categories
 - example: $|H| \approx 200$ from Attributes.csv

Calculation Example

Using:

$$\varepsilon = 0.10, \delta = 0.05, |H| = 200$$

we get:

$$\begin{aligned} m &\geq \frac{1}{0.10^2} \ln \left(\frac{200}{0.05} \right) \\ m &\geq 100 \cdot \ln(4000) \\ m &\geq 100 \cdot 8.29 \approx 829 \end{aligned}$$

So the theoretical minimum is:

$$m \approx 830$$

Interpretation:

This means the system would need at least **about 830 labeled examples** to satisfy this theoretical bound under the given assumptions.

Target	Sample Count
Theoretical minimum (PAC bound)	~830 total labeled examples
Per attribute category minimum	4 - 5 samples
Engineering recommendation per category	20 - 50 samples
First-phase collection target	200 labeled supplier submission examples

5. Data Requirements from eParts

Context: The CSV files already shared (Attributes, Manufacturers, Product_attribute_values, etc.) give us the correct output side — they show what correctly mapped attribute values look like. What we still need is the input side: the raw supplier submissions (spec sheets, catalog rows, email text) paired with the correct attribute mappings that a person has already verified.

Priority 1-A — Labeled Supplier Submission Examples [BLOCKING]

What we need: Raw supplier text (extracted from PDF spec sheets, CSV rows, or email bodies) paired with the verified correct attribute mappings. Minimum quantity: 200 examples. To get **95% statistical confidence upward boundary we need 830 examples per attributes in calculation.**

The examples below are based on the actual supplier PDF files shared in the data folder.

Example 1 — From a supplier PDF spec sheet (Automation Components, Inc. AIM2):

Field	Value
Submission ID	SUB-2025-0047
Supplier	Automation Components, Inc.
Source type	PDF spec sheet
Extracted text	AIM2 — Analog Input to Optically Isolated Analog Output Supply Voltage: 24 VAC (+/- 10%), 50/60 Hz Supply Current: 200 mA maximum Input Voltage Signal Preset Ranges: 0-5V, 0-10V, 0-15V, 0-20V Input Current Signal Preset Ranges: 0-20 mA, 4-20 mA
Verified attribute mappings	SUPPLY VOLTAGE (ID=29) → '24VAC' INPUT_SIGNAL (ID=51) → '0-10V' (primary range) OUTPUT_SIGNAL (ID=16) → '0-10V' CURRENT RATING (ID=5) → '200mA'
Confidence label	HIGH — all fields clearly stated in source text

Example 2 — From a supplier PDF datasheet (Accuenergy AcuCT Flex RCT16-2500):

Field	Value
Submission ID	SUB-2025-0061
Supplier	Accuenergy
Source type	PDF datasheet
Extracted text	AcuCT Flex Series Rogowski Coil CT Current Range: 5A – 50,000A Accuracy: 0.5% — IEEE C57.13 class 0.6 Operating Temperature: -20°C to 70°C Operating Humidity: Non-condensing, 0 to 95% RH Frequency Range: 45Hz – 65Hz
Verified attribute mappings	CURRENT RATING (ID=5) → '5A TO 50000A' ACCURACY (ID=46) → '0.5%' TEMPERATURE RANGE (ID=6) → '-20 TO 70°C' IMPEDANCE (ID=3) → [not stated — leave blank]
Confidence label	HIGH

Example 3 — Ambiguous case (CSV row, should be routed to review):

Field	Value
Submission ID	SUB-2025-0089
Supplier	[Supplier Name]
Source type	CSV row
Extracted text	'Duct sensor, 10k, 2-wire, -40 to 250F'
Ambiguity issue	'10k' could match: • 10K-3 THERMISTOR (Attribute_ID=42, sim=0.74) • 10K-2 THERMISTOR (Attribute_ID=42, sim=0.71) • 10K Type II (Attribute_ID=42, sim=0.69)
Reason code	AMBIGUOUS_VALUE
Confidence label	LOW — route to Human Review Queue
Reviewer action	Confirm which 10k thermistor type from supplier documentation.

Where to source these examples: Any past supplier submission that was manually manually processed by the eParts team — email attachments, SFTP drops, or catalog imports where a staff member looked up the product and entered it into PIMS. Even 50 real examples are enough to start building the rule engine; 200 enables the ML layer.

Priority 1-B — Products Master Table (with Descriptions) [BLOCKING]

What we need: A table linking Product_ID to part number, name, free-text description, and manufacturer. Currently we have Product_attribute_values (e.g., Product 294 has ELEMENT = '3K THERMISTOR') but no product name or readable description. This is the reference library the similarity model searches against.

Requested format:

Product_ID	PartNumber	Product_Name	Description (free text)	Manufacturer_ID
294	[part no.]	ACI Strap-On Temperature Sensor	Strap-on thermistor, 3K resistance, -40 to 250°F range, 2-wire.	[Mfr_ID]
[AIM2 ID]	AIM2	ACI Analog Isolation Module	Analog input to isolated analog output, 24VAC supply, 0-10V / 4-20mA ranges.	[Mfr_ID]

Note: Description text does not need to be polished — spec sheet language or concatenated attribute values from the existing eParts database are acceptable.

Priority 1-C — Staging / Canonical Table Schema [BLOCKING]

What we need: The column definitions for the Intermediate Structured Layer (defined in Section 6.3 of the eParts architecture requirements). This is the table that the Attribute Prediction Service reads from.

Specifically, we need to know:

- What column names does a canonical row contain?
- Which columns are always populated vs. optional?
- How are multi-value fields (e.g., multiple voltage ranges) represented?
- What format are numeric ranges stored in?

Example of what we are asking for (illustrative — please provide the actual schema):

Column Name	Type	Example Value	Always Present?
submission_id	string	SUB-2025-0047	Yes
supplier_id	int	18	Yes
raw_product_name	string	AIM2	Yes
raw_description	string	Analog Input to...	Yes
supply_voltage_raw	string	24 VAC (+/- 10%)	No
input_signal_raw	string	0-10V, 4-20mA	No
temperature_raw	string	-20C to 70C	No
source_channel	string	PDF / CSV / Email	Yes
ingested_at	datetime	2025-11-14 09:32:00	Yes

Priority 2-A — Valid Values per Attribute [Highly Recommended]

What we need: For each attribute, all accepted values. Without this, the model may predict values that do not exist in the eParts system.

Attribute_ID	Attribute_Name	All Valid Values
16	OUTPUT_SIGNAL	RESISTANCE, 0-10V, 4-20mA, 0-5V, DIGITAL, PWM, 0-20mA, 0-1mA
5	CURRENT RATING	Numeric — accepted format: '4-20mA', '200mA max'
6	TEMPERATURE RANGE	Numeric range — format: '-20 TO 70°C'
42	ELEMENT	3K THERMISTOR, 10K-3 THERMISTOR, 100K THERMISTOR, 100 OHM PLATINUM RTD, 1000 OHM PLATINUM RTD
66	MOUNTING LOCATION	STRAP-ON, DUCT, WALL, IMMERSION, AVERAGING, WELL
46	ACCURACY	Numeric — format: '0.5%', '±1%'

Priority 2-B — Historical Error and Ambiguous Cases [Highly Recommended]

What we need: Cases where a human reviewer had to correct the system or make a judgment call. These are the most valuable training examples for teaching the model to recognize the limits of its own confidence.

Error Case #001:

Field	Details
Extracted text	'AcuCT-0750-333 current transformer, 750A, 333mV output'
Wrong prediction	OUTPUT_SIGNAL = '0-10V' (confidence was 0.82 —

	too high)
Correct value	OUTPUT_SIGNAL = '333mV'
Why it was wrong	'333mV' is a non-standard output range not well represented in training data. Model defaulted to the most common value.
Action taken	Added to training set; rule added for 'mV output' pattern.

Error Case #002:

Field	Details
Extracted text	'Flex Rogowski, 36-inch coil, 5000A'
Wrong prediction	ELEMENT = '10K-3 THERMISTOR' (wrong category entirely)
Correct value	ELEMENT does not apply; correct attribute is CURRENT RATING.
Why it was wrong	Short, sparse description — not enough context for the model.
Action taken	Flagged as ambiguous; requires minimum field count before ML prediction is attempted.

Minimum quantity: 50 error cases to start.

Priority 2-C — Read Access to Existing MSSQL / PostgreSQL Reference Data

The eParts architecture (§6.10) already calls for reading reference data from the existing SQL Server for normalization and validation. Granting the ML service read-only access to the relevant tables would allow:

- Real-time part number lookups during rule engine matching.
- Manufacturer name normalization (e.g., 'ACI' → 'Automation Components, Inc.').
- Valid-value validation at prediction time.

We only need read access to reference/lookup tables — never to transaction or customer data. This aligns with the architecture requirement to 'keep read load controlled so we do not hurt production systems.'

Priority 3 — Additional Helpful Data

Supplier Submission Templates: If suppliers use a standard CSV template or form (e.g., a fixed-column spreadsheet from a specific distributor), sharing those templates lets us write targeted parsing rules that significantly improve rule engine coverage for that supplier.

Request Volume Distribution by Product Category: A rough count of how many submissions per month fall into each product category (sensors, actuators, valves, relays, current transformers, etc.). This tells us where to prioritize training data collection so the model performs best on the most frequent product categories.

Known Problematic Suppliers or File Formats: If certain suppliers consistently send files that cause parsing issues, knowing this in advance lets us build targeted pre-processing before the ML

layer even sees the data. This directly supports the architecture requirement to 'handle messy inputs safely.'

5.6 Summary of Data Requests

Priority	Data Type	Minimum / Notes
P1 — Blocking	Labeled Supplier Submission Examples	200 input-output pairs
P1 — Blocking	Products Master Table (with descriptions)	Full active catalog
P1 — Blocking	Staging / Canonical Table Schema	Column definitions + example row
P2 — Recommended	Valid Values per Attribute	All active attributes
P2 — Recommended	Historical Error / Ambiguous Cases	50+ cases
P2 — Recommended	Read Access to MSSQL Reference Tables	Read-only, lookup tables only
P3 — Helpful	Supplier Submission Templates	Any standard formats in use
P3 — Helpful	Volume by Product Category	Rough monthly counts
P3 — Helpful	Known Problematic Suppliers / Formats	List with issue description

Data privacy note: Supplier submissions may contain commercially sensitive information. Anonymized or redacted versions are fully acceptable — supplier names, pricing, and contact details can be removed. We only need the product specification text and the corresponding correct attribute mappings. All data will be used solely for training and will not be shared externally.

6. Alignment with the eParts Architecture Requirements

The following table maps the ML service design decisions to the specific requirements in the eParts architecture document.

Architecture Requirement (§)	Requirement Text	How This Is Addressed
§6.4	Retraining must be manually triggered	Controlled retraining cycle with ops lead approval and model registry promotion (Section 4.8)
§6.4	Predictions must be reproducible	Every prediction record stores the model version ID; same version + same input always gives same output
§6.4	Track model quality drift over time	Metrics tracked per model version: auto-accept rate, correction rate, confidence distribution
§6.5	Keep decisions explainable to	Reason codes

	ops users	(RULE_NO_MATCH, LOW_SIMILARITY, AMBIGUOUS_VALUE, MISSING_FIELD) shown per item (Section 4.6)
§6.5	Allow ops leads to adjust thresholds	Thresholds configurable; every change logged with who, when, and new value
§6.3	Store parsing outputs separately from predictions	ML layer reads from Staging Table; predictions written to separate column set
§6.3	Track lineage end-to-end	Every record carries submission_id from raw file through to PIMS writeback
§6.8	Audit trail: who changed what, when	All reviewer decisions and model predictions stored with timestamp and model version
§6.1	Handle messy inputs safely	Rule engine validates fields before ML layer; malformed inputs return MISSING_FIELD reason code, do not crash pipeline

7. Proposed Next Steps

Step	Action	Owner
1	eParts confirms which P1 data is available and in what format	eParts
2	eParts shares Staging Table schema + 1 example row	eParts
3	eParts shares Products Master Table	eParts
4	eParts shares 200 labeled supplier submission examples (P1-A)	eParts
5	Team builds rule engine and tests on provided data	Studio Team
6	Team trains similarity model; evaluates on held-out examples	Studio Team
7	First end-to-end demo: supplier PDF in → attribute predictions + confidence out	Both
8	Human Review Queue tested with ops users	Both
9	First retraining cycle after initial reviewer feedback	Studio Team + eParts sign-off

Document prepared by MSE Studio 2026 — eParts Services Project Team
For questions about this document, please contact the project team.